After we finished handling the Data layer (services, repository, and models) and then created the Cubit and provided it using BlocProvider, now....and FINALLYYYY ... we will start connecting them together to see our work coming to life

Data Layer <=> Business Logic Layer <=> Presentation Layer

1- Building the connection between the UI and the Cubit using BlocBuilder

Now we have::

- the data ready
- the repository ready
- the cubit ready
- and BlocProvider providing the cubit widget tree

The flow now is:
UI requests data => Cubit handles the logic => State changes => BlocBuilder notices the change => UI rebuilds with new data

So when the cubit retrieves data from the repository and emits a new state, BlocBuilder immediately reacts and rebuilds the widgets depending on the current state

Remember when I said the UI is "the thing with no brain"
It only listens and display

The Cubit is the one doing the thinking:

- Did the request succeed?
- Did an error happen?
- Is the data still loading?

Then BlocBuilder listens to those state changes and updates the screen according to that

2- Building the actual UI and handling the different states

After connecting the Cubit to the UI using BlocBuilder, we can finally build the actual screen and display the data to the user 💃 💃

note: Data coming from APIs is not always instantly available

The data can be:

- Loading
- Loaded
- Error happened

So instead of building one static screen, we build the UI depending on the current state
For example:
If the data is still loading:
.... show a loading indicator

If the data is loaded successfully:
.... display the list of data

If an error happened:
.... display an error message

the screen automaticlly changes depending on the state emitted from the Cubit
So instead of manualy controlling everything, the state itself controls what the user sees

3- Search filtering logic

we keep another list specifically for the filtered results.

Why is that?
that's a good question....Because we don't want to modify the original data list.

So we have:

- Original list → contains all data
- Filtered list → contains only matching search results

Now when the user types inside the search bar, something very important happens:
the search updates letter by letter.

ya3ni eeeh?
that means that every single character typed triggers a new filtering process.

So the flow becomes:

User types a letter
-> onChanged detects the new text
-> clear the old filtered list
-> search again through the original data

-> add matching items to the filtered list
-> rebuild the UI with the new filtered results

This process repeats continuously with every character change.

Turning the AppBar into a Search Bar

there is a cute trick when it comes to the search bar....
Instead of navigating to a completely different screen for searching, we make the AppBar itself transform into a search bar when pressing the search icon..... what an idea !!

But internally, Flutter handles this almost like opening a new screen.... like we're tricking flutter or something

Because Flutter already has default navigation behaviors:

- automatic back button handling
- closing search when pressing back
- restoring the original AppBar

So instead of manually controlling all these behaviors ourselves, we take advantage of Flutter's built-in navigation system ..... what an easy way, am i right ?

The process becomes something like this:

User presses search icon
->Flutter opens the search mode
->AppBar transforms into a search field
->user types search query
->search filtering updates live
->pressing back exits search mode
->original AppBar returns automatically

This is actually a smart design choice because it allows us to reuse Flutter's default navigation behavior instead of reinventing everything manually

So overall, these videos were mainly about finally connecting all the layers together and making the application reactive:

- retrieving data from the Cubit
- rebuilding UI based on states
- displaying proper UI for loading/error/success
- implementing live search filtering

- and creating dynamic AppBar search behavior